Resumen de la semana 4

Interrupciones externas en el ESP32

1. Sondeo frente a interrupción

Con **sondeo (polling)**, el programa revisa el estado de un pin en cada vuelta de `loop()`. Si está ocupado en una tarea larga, por ejemplo `delay(2000)`, una pulsación breve puede ocurrir y terminar antes de que vuelva a leerla.

Con una **interrupción externa**, el periférico GPIO detecta el cambio de la señal y avisa a la CPU. El programa hace una pausa corta, ejecuta la **ISR** (rutina de servicio de interrupción) y luego retoma lo que estaba haciendo.



2. Flancos y pines del ESP32

Un **nivel** indica si una señal está en HIGH o LOW. Un **flanco** es el momento del cambio. Con `INPUT_PULLUP`, el pin normalmente está en HIGH y pasa a LOW al presionar el botón.

Modo	Transición	En un botón con pull-up
`FALLING`	1 → 0	Al presionar
`RISING`	0 → 1	Al soltar
`CHANGE`	Ambas	Al presionar y al soltar

Para la alarma de clase se utiliza **GPIO21 con** `INPUT_PULLUP`. También son adecuados GPIO 16, 17, 18, 19, 22, 23, 25, 26, 27, 32 y 33. Los GPIO 34-39 pueden detectar interrupciones, pero necesitan una resistencia pull-up externa. Se evitan los pines de strapping y los GPIO 6-11, dedicados a la flash.

3. `attachInterrupt()` y la ISR

La interrupción se configura una sola vez en `setup()`. Primero se define el pin con `pinMode()` y después se registra la función que atenderá el evento. La instrucción principal es:

```
pinMode(21, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(21),
                isrBoton, FALLING);
```

`digitalPinToInterrupt(21)` indica el pin, `isrBoton` es el nombre de la función (sin paréntesis) y `FALLING` indica el cambio que la activa. La ISR no se llama manualmente; se ejecuta cuando llega el aviso del hardware.

La ISR debe ser **muy corta**. Se permite, por ejemplo, marcar una bandera o registrar un tiempo. No conviene usar dentro `Serial.println()`, `delay()`, esperas o cálculos largos. Si la ISR se bloquea, el **watchdog** puede reiniciar el ESP32 y aparecer un mensaje como “Guru Meditation Error”.

4. `IRAM_ATTR`, `volatile` y patrón bandera

La **flash** guarda el programa, pero a veces la caché de lectura no está disponible. `IRAM_ATTR` coloca la rutina crítica en la RAM interna de instrucciones (IRAM) para poder atender el evento con más seguridad.

Cuando una variable puede ser cambiada desde una ISR y consultada por `loop()`, se declara `volatile`. Así, el compilador no supone que su valor permanece fijo. Esto **no convierte una operación en atómica**: sigue siendo necesario pensar cómo se comparten los datos.

```
volatile bool alerta = false;

void IRAM_ATTR isrBoton() {
  alerta = true;
}

void loop() {
  if (alerta) {
    alerta = false;
    Serial.println("Alerta detectada");
  }
}
```

El patrón bandera divide responsabilidades: **la ISR avisa y termina**, mientras que `loop()` borra la bandera y ejecuta la acción que tardaría más. Una bandera booleana solo indica que pasó algo: si hay tres eventos antes de atenderla, pueden quedar representados como uno. Cuando se necesita el número de eventos se utiliza un contador, con la debida precaución al compartirlo.

5. El rebote de un pulsador

Al presionar un botón mecánico, sus contactos pueden abrirse y cerrarse varias veces en pocos milisegundos. Si se atiende cada `FALLING`, una sola pulsación puede disparar varias ISR. El **antirrebote** de la clase descarta flancos ocurridos antes de **50 ms** desde el último aceptado.

```
const uint8_t PIN_BOTON = 21;
const uint8_t PIN_LED = 19;
const uint32_t T_REBOTE = 50;
volatile bool alerta = false;
uint32_t tUltimo = 0;

void IRAM_ATTR isrBoton() {
  uint32_t ahora = millis();
  if (ahora - tUltimo >= T_REBOTE) {
    alerta = true;
    tUltimo = ahora;
  }
}
```

La comparación `ahora - tUltimo >= 50` es la misma idea de resta de tiempos trabajada en la semana 2. `tUltimo` solo lo utiliza la ISR en este ejemplo; la variable compartida con `loop()` es `alerta`.

6. Ejemplo completo de la alarma

Continuando el fragmento anterior, el ejemplo de las diapositivas configura un LED en GPIO19 y un botón en GPIO21. Al presionar el botón se marca la alerta. `loop()` cambia el estado del LED y escribe un mensaje, incluso si estaba ocupado en una espera:

```
void setup() {
  Serial.begin(115200);
  pinMode(PIN_LED, OUTPUT);
  pinMode(PIN_BOTON, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(PIN_BOTON),
                  isrBoton, FALLING);
}

void loop() {
  if (alerta) {
    alerta = false;
    digitalWrite(PIN_LED, !digitalRead(PIN_LED));
    Serial.println("Alerta! Puerta abierta");
  }
  delay(2000);
}
```

El `delay(2000)` se mantiene aquí para mostrar la diferencia con el sondeo: aunque el programa principal esté ocupado, la ISR puede levantar la bandera. La respuesta visible ocurrirá cuando `loop()` vuelva a atenderla.

7. Preguntas de repaso resueltas

1. ¿Qué flanco se elige para una alarma con `INPUT_PULLUP`**? FALLING**: el pin pasa de 1 a 0 al presionar el botón.

2. ¿Cuáles llevan `volatile`**?** `pulsos`, si la ISR lo incrementa y `loop()` lo lee. `tUltimo` no, si solo lo utiliza la ISR; `total` tampoco, si solo lo usa `loop()`.

3. ¿Por qué se reinicia si se hace `Serial.println()` **dentro de la ISR?** Porque esa operación puede tardar o bloquear la atención; hay que sacar el mensaje de la ISR y ejecutarlo desde `loop()` usando una bandera.

4. Se presiona diez veces sin antirrebote. ¿Cuántos avisos? Normalmente **más de diez**, porque cada pulsación puede generar varios flancos por el rebote. No se puede garantizar una cifra exacta.

5. Timer a 1 MHz con alarma 500 000 y autorrecarga. `t = 500 000 / 1 000 000 = 0,5 s` por aviso; `1 / 0,5 = 2` interrupciones por segundo.

> **Para recordar:** un buen programa con interrupciones decide el flanco correcto, usa un GPIO adecuado, mantiene la ISR corta, marca variables compartidas con `volatile`, coloca la ISR en IRAM y controla el rebote del botón.